

PRACTICE EXERCISES



Module 7: NoSQL & Redis

Module 7 - 10 Hours

NoSQL & Redis

Instructions

You need Redis running, the `icicibank` PostgreSQL database loaded, and `code/icici_cache.js` available.

Level	Meaning	Expected time
Level 1	Recall and understanding	35 min
Level 2	Apply. Build cache logic and measure it.	80 min
Level 3	Judgement. Decide what to cache and what not to.	40 min

Rules:

- Every measurement answer must show the numbers you actually got, not the ones in the lab guide.
- Any code you write goes in a file named `M07_<yourname>.js` and must run under Node 18+.
- Prefix every key you create with `icici:ex:` so the cache can be cleaned up.

How much to attempt

The questions marked **(Core)** are the required set. They fit the time above, and together they cover every objective of the module. Everything unmarked is **stretch work**: attempt it if you finish early, or take it home.

You are not behind if you have completed only the core set.

Level	Core set	Stretch
Level 1	Q1, Q2, Q3, Q5, Q6, Q8, Q9, Q10	Q4, Q7, Q11, Q12
Level 2	Q13, Q14, Q16, Q17, Q19, Q20, Q22, Q23	Q15, Q18, Q21, Q24
Level 3	Q25, Q26, Q27	Q28, Q29, Q30

Level 1 - Understanding

Q1. (Core) Explain what NoSQL means, name the four families, and give one ICICI Bank workload that fits each.

Q2. (Core) State the CAP theorem in one sentence, then explain why "choose two of three" is a misleading way to teach it. For the core banking ledger, the balance cache and the session store, say which property each gives up and why that is the right choice.

Q3. (Core) Give three reasons Redis must not be used as the system of record for account balances. For each, describe the specific failure that would follow.

Q4. Redis executes commands one at a time on a single thread. State one advantage and one danger of that design, with a concrete banking example of each.

Q5. (Core) For each requirement, name the Redis data structure you would use and the commands involved.

1. The last ten transactions on an account, newest first, never more than ten.
2. Whether a card is on the blocked list, answered in constant time.
3. The three highest balances in a branch, without sorting.
4. A count of how many times the balance endpoint was called today.
5. A customer's session, where one field is updated without rewriting the rest.
6. No more than five OTP requests per customer per hour, on a sliding window.
7. The nearest ATM within two kilometres.

Q6. (Core) Explain the difference between cache-aside and write-through in terms of when the cache is populated, what a write costs, and what the first read after a write returns.

Q7. Explain why write-behind must not be used for account balances. Describe the exact sequence of events that loses money, and name one workload where write-behind is legitimate.

Q8. (Core) Two rules govern invalidation: delete rather than overwrite, and delete after the commit rather than before. Explain the specific bug that each rule prevents.

Q9. (Core) A TTL is described as a backstop. Explain what it is a backstop for, given that invalidation has already been implemented correctly.

Q10. (Core) Describe a cache stampede, name three ways to prevent one, and say which you would implement first and why.

Q11. `maxmemory` is unset on a production Redis. State what will eventually happen and why it is worse than the alternative. Then explain the trap in the `volatile-lru` policy.

Q12. A balance cache is reporting a 40 percent hit rate. List four possible causes and say how you would tell them apart.

Level 2 - Build and measure

Data structures

Q13. (Core) Using `redis-cli` only, build a mini-statement for account 9010 containing five entries, capped so that pushing a sixth discards the oldest. Show the commands and the final `LRange`.

Q14. (Core) Build a sorted set of every account balance in branch 5, then answer three questions with single commands: the top two accounts, the rank of account 9015, and how many accounts hold more than 200,000.

Q15. Implement a "recently viewed accounts" feature for a relationship manager: the last five accounts they opened, no duplicates, most recent first. State which structure you chose and why the obvious one does not work.

Caching

Q16. (Core) Write `getCustomerProfile(db, customerId)` using cache-aside over a Redis hash, caching first name, last name, city, KYC status and risk rating with a one-hour TTL. Show a miss followed by a hit.

Q17. (Core) Extend `icici_cache.js` with `getBranchSummary(db, branchId)` that caches the customer count, account count and total deposits for a branch, with a ten-minute TTL. Measure the time for a cold call and a warm call.

Q18. Write `invalidateCustomer(customerId)` that removes every cached key belonging to one customer, including their accounts. Explain how you find the account keys, and why `KEYS icici:*` is not an acceptable answer.

Q19. (Core) Benchmark `get_branch_summary` over 500 calls across the 8 branches, with and without the cache. Report elapsed time, database calls and hit rate for each.

Q20. (Core) Demonstrate the stale-cache bug on `get_customer_profile`: cache a profile, change the city directly in the database, and show that the cached value is served. Then fix it and prove the fix.

Q21. Write a function that caches the result of an expensive query - the branch performance pack from Module 3 - with a 15-minute TTL and jitter. Measure cold and warm timings and state the ratio.

Sessions and limits

Q22. (Core) Extend the session store so a session records the IP address and the last-seen timestamp, and add `active_sessions(customer_id)` returning every live session with its remaining TTL. Show it working with three devices.

Q23. (Core) Implement a login rate limit: five failed attempts per account per fifteen minutes, after which the account is locked for thirty minutes. Show the sequence of six attempts and the lock taking effect.

Q24. Implement `is_card_blocked(card_id)` backed by a Redis set, with a function that loads the set from the database on startup and a function that adds a card at runtime. Explain what happens if Redis is unavailable, and what your code does about it.

Level 3 - Judgement

Q25. (Core) For each of these, decide whether to cache it, and give the TTL or the reason not to.

1. A customer's account balance.
2. The result of a fund transfer.
3. The bank's list of branches and IFSC codes.
4. Today's foreign exchange rate for USD to INR.
5. A customer's full transaction history for the last seven years.
6. Whether a card is blocked.
7. A one-time password that was just sent.
8. The number of failed login attempts on an account.

Q26. (Core) The mobile team proposes caching the balance with a 24-hour TTL, "because balances rarely change and it would give us a 99.9 percent hit rate". Write your response. Include what you would ask them, what you would propose instead, and how you would decide the number.

Q27. (Core) Redis becomes unavailable at 09:15 on a Monday. Describe what happens to the balance endpoint, the session store and the rate limiter, given the code in this module. For each, say whether the behaviour is acceptable and what you would change.

Q28. A developer proposes storing the account balance only in Redis, with a nightly job writing it to the database, "because Redis is faster and the database is just for reporting now". Write the response you would send. Be specific about what is lost.

Q29. The cache invalidation in `transfer()` only covers transfers made through the application. Name three other paths that change a balance and would leave the cache stale, and describe an architecture that would catch all of them. State the cost of that architecture.

Q30. The balance cache has a 96 percent hit rate. The Head of Digital Channels asks whether the remaining 4 percent can be eliminated by pre-loading every account at startup. Give your answer with reasoning, including what pre-loading would cost and when it would be the right choice.

Q31. (Written, 10 to 15 lines.) Reply to ticket ICICI-4701. State the measured reduction in database load, explain to a non-technical reader why keeping a copy of the answer is safe when the copy might be out of date, describe the one thing that could go wrong and what limits the damage, and confirm what has changed for the internet banking session problem. Do not use the words "cache-aside", "TTL" or "invalidation" - explain the ideas instead.